ASD – Software architecture

Parallel programming part 2



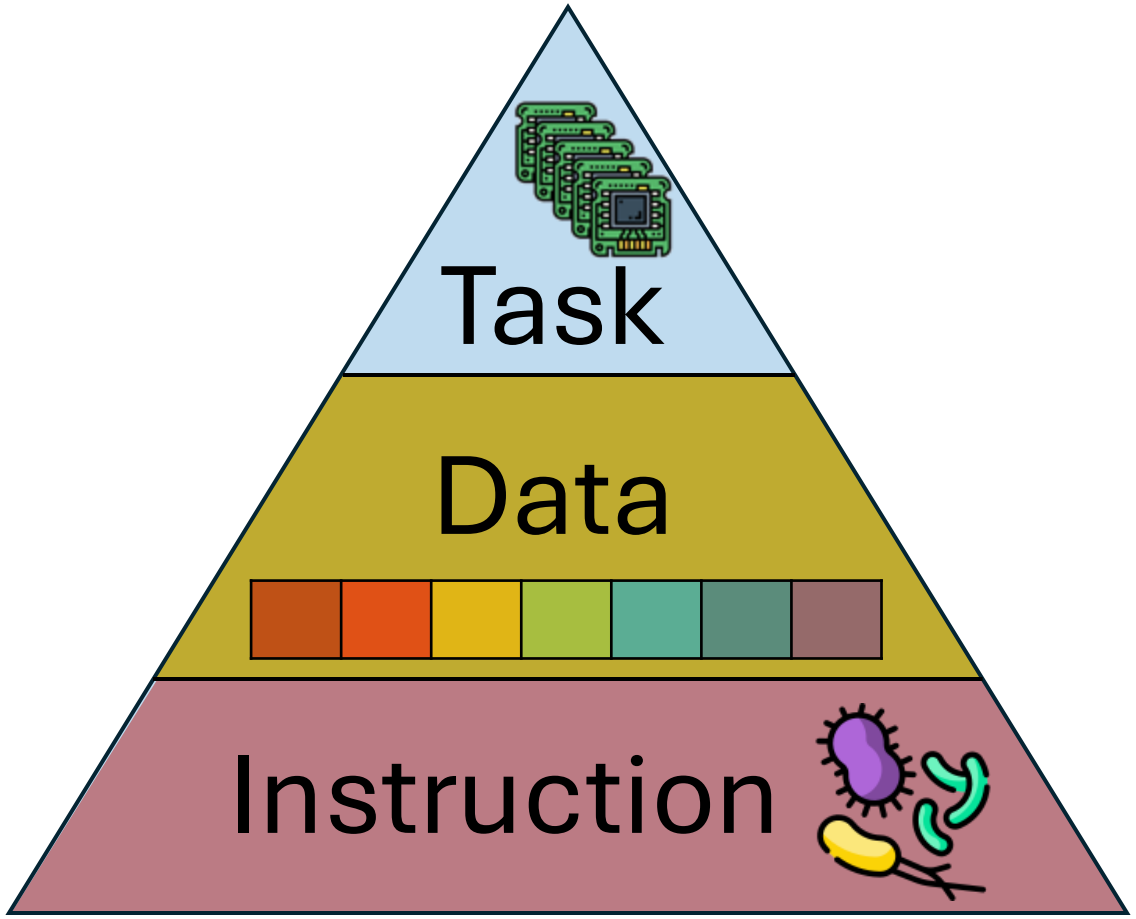


Parallel hierarchy…

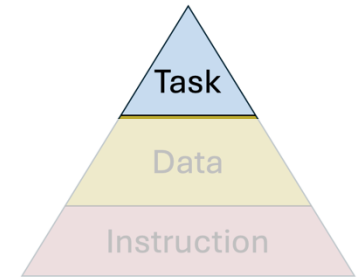
Task parallelism

Data parallelism

Instruction parallelism



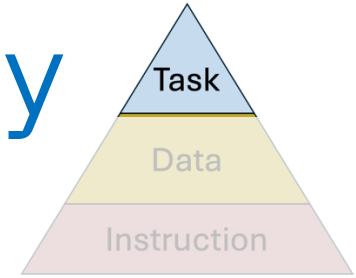
Task parallelism



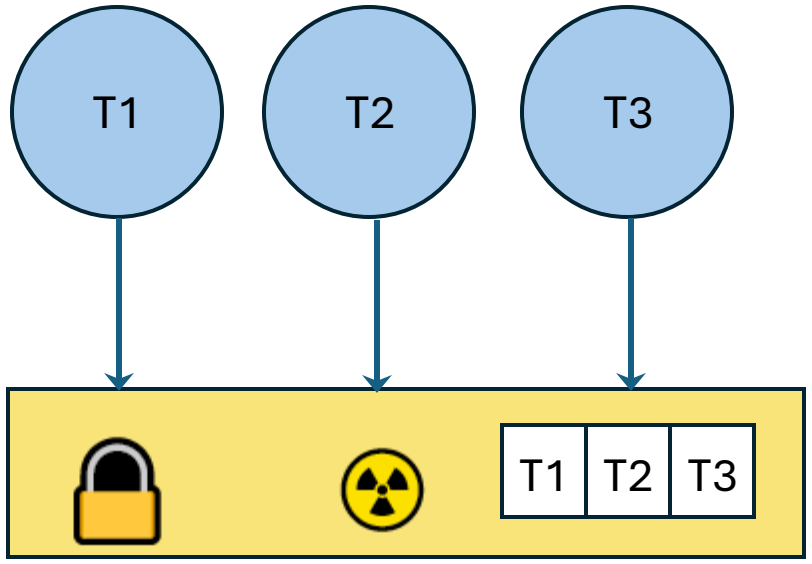
Tasks ≈ ‘Independent programs’

- Each task has its own data
  - Memory explosion
- **Communication** between tasks
  - Time consuming
  - Complex
  - Hard to prove **correctness**
  - No easy solution
  - Majority! of published research is incorrect

Distributed vs shared memory



Shared memory

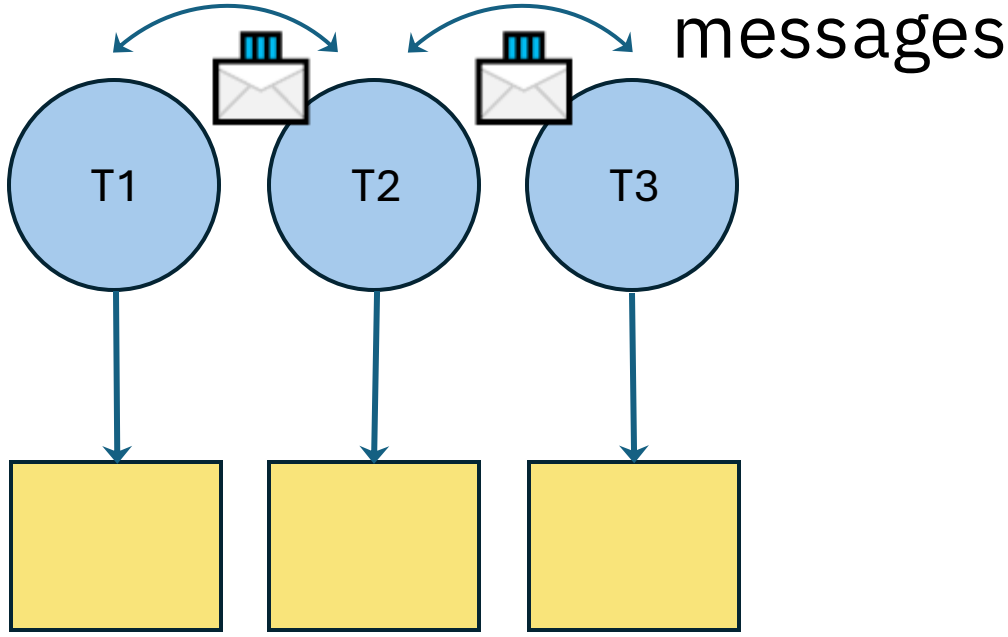


locks / atomics / partition

+ fast

– complex

Distributed memory

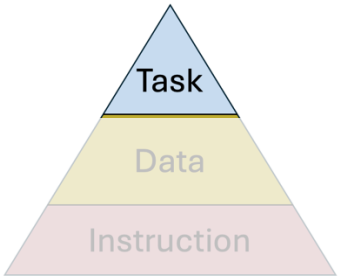


MPI: Message passing interface

– slow

+ provably correct

Shared memory tasks

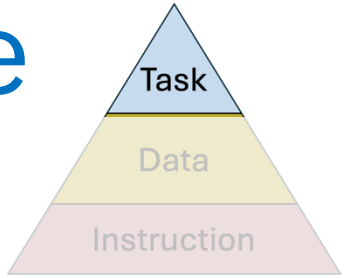


Communication happens using data structures

- queue in → | A | B | C | D | E | → out
- stack | A | B | C | D | E | ⇆ in/out
- tree

More complex structures = too much pain

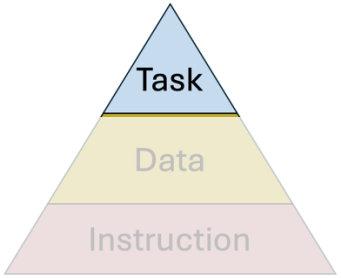
Locked/lock free/wait free



Shared memory access can be deconflicted

- **Locked**: one task locks, all other wait
  - + Easy to implement, efficient for locking task
  - – All other tasks must idle during lock
- **Lock free**: tasks only lock elements they need
  - + Other tasks can claim non-locked elements
  - – coordination extremely subtle and complex
- **Wait free**: lock free + other tasks can **help** locking task
  - + Reduces contention on ‘popular’ elements
  - – The most complex subject in all of CS

Locked/lock free/wait free



Why different mechanisms?

- **Locked**: one task locks, all other wait
  - if 2 tasks lock, program hangs = **deadlock**
  - if 1 task keeps locking, other tasks **starve**
- **Lock-free**: tasks only lock elements they need
  - \+ one task will always make progress
  - other tasks can still **starve** or **livelock**
- **Wait-free**: lock-free + other tasks can **help** locking task
  - \+ locking task will always succeed in *fixed number of steps*
  - \+ other tasks do not starve or livelock
  - high fixed overhead

Guarantees

Locked

- 1 thread can deadlock system

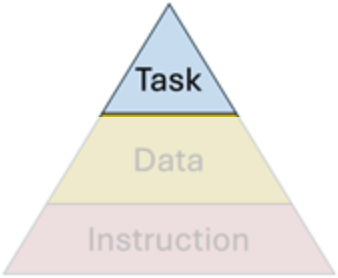
Lock free

- No one thread can deadlock system,
  but no guarantee on wait times

Wait free

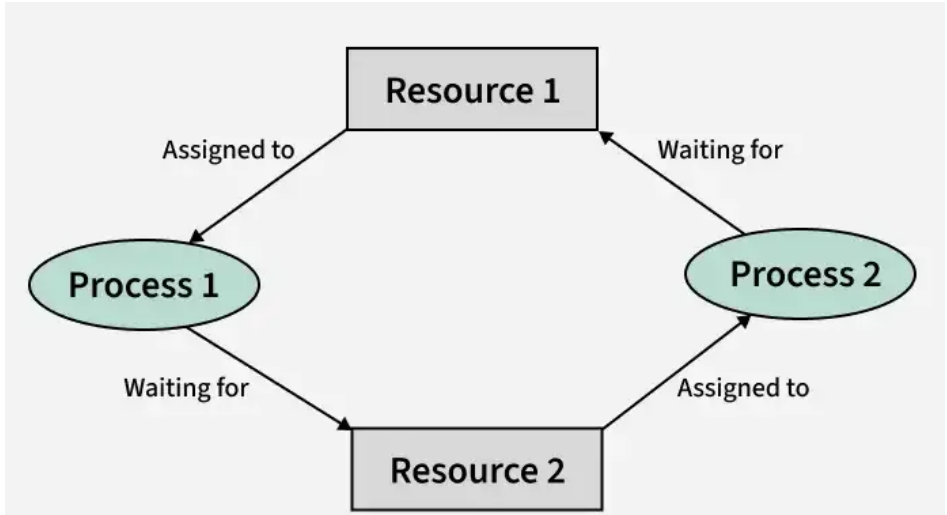
- No thread has to wait more than X steps (***steps***, not time)

Deadlock/starvation



Deadlock:

- task A locks A, needs B
- tasks B lock B, needs A

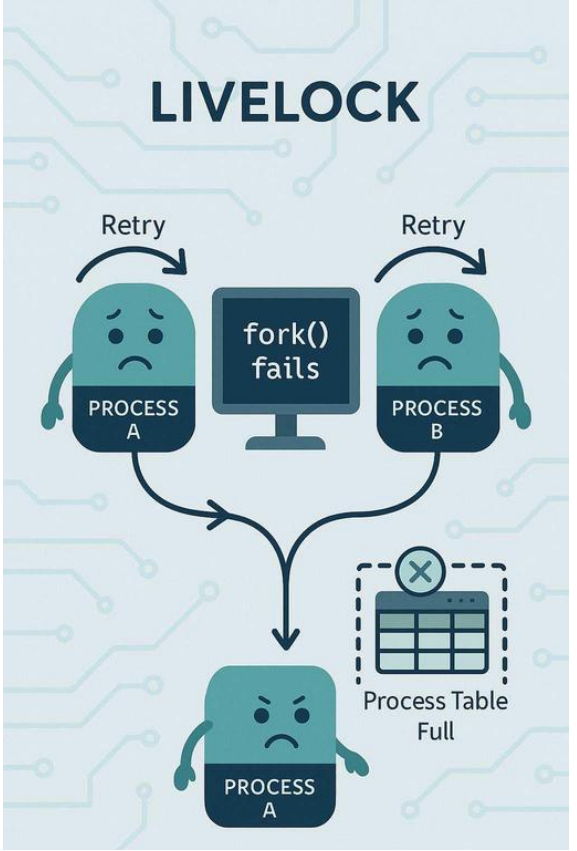


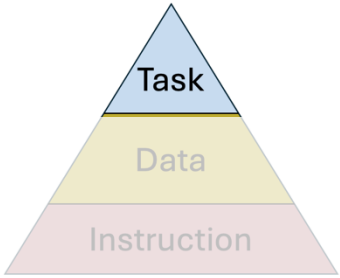
Starvation:

- NS task keeps winning
- EW task cannot get lock



Livelock





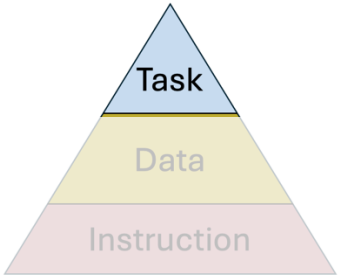
Livelock:

- no tasks wait
- no task can make progress

T1,T2:

```
try //livelock for T1,T2 if mem full
   while (memfree() < 100K) { wait(); }
   a = AllocateMem(100K) //locks
finally
   a.free()
```

How do locks work?



Out of scope for this course, but

- All processors have **special hardware** for locks
- This guarantees that only one task can get a lock
  - other tasks are made to wait
- processors also have support for micro-operations: **atomics**
  - T1, T2: `old = atomicAdd(x, 1);`
    - Before: x == 0, After: x == 2;
    - After: T1.old == 0 **or** 1, T2.old == 1 **or** 0, T1+T2.old == 1
    - never: T1.old == 1 **and** T2.old == 1
- Atomics can build lock-less parallel data structures
  - E.g.: queue, but **not** stack

ASD – Software architecture

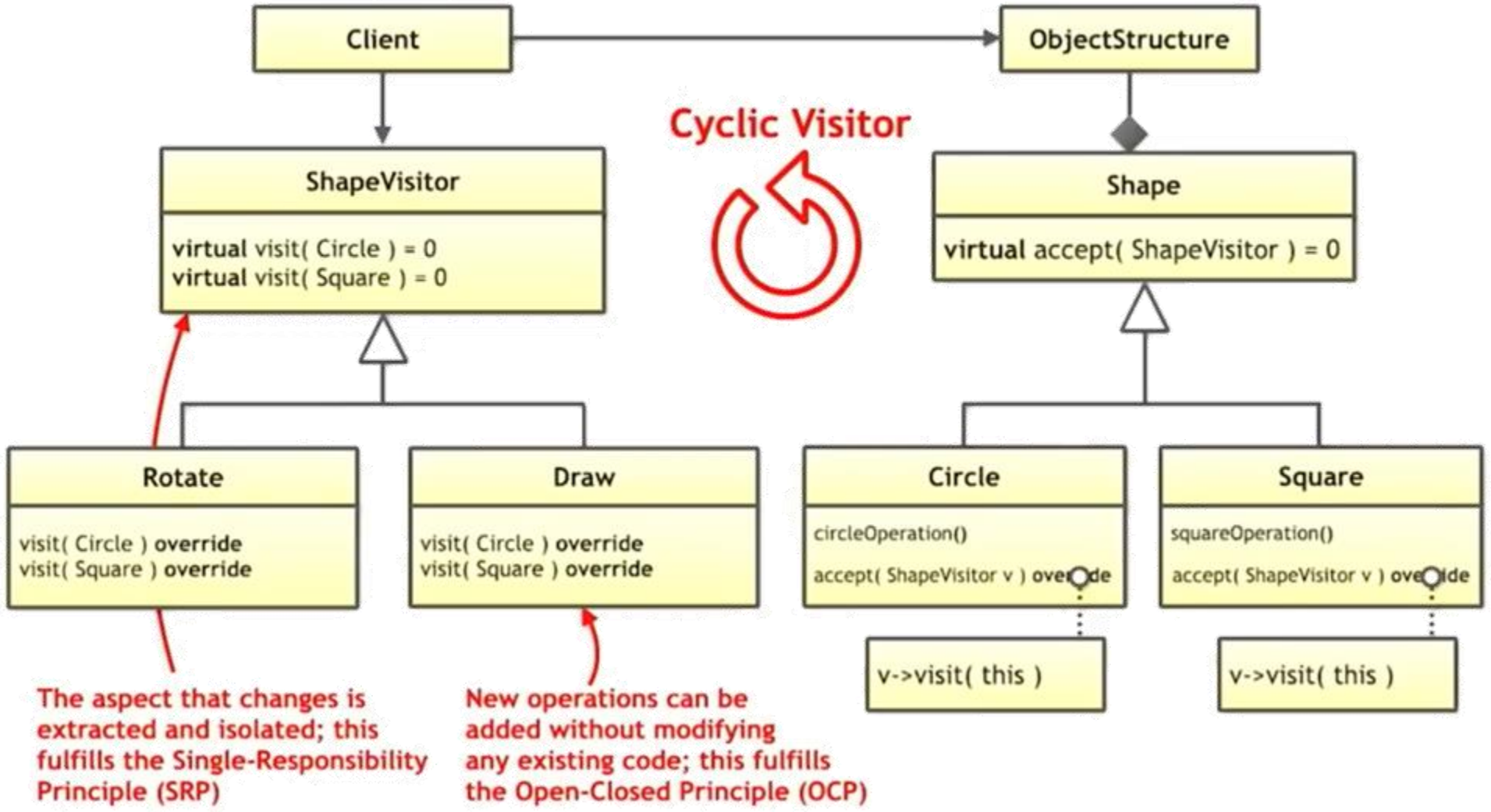
Aspect oriented programming





Why AOP (Aspect oriented programming)?

Remember our visit pattern



Why AOP?

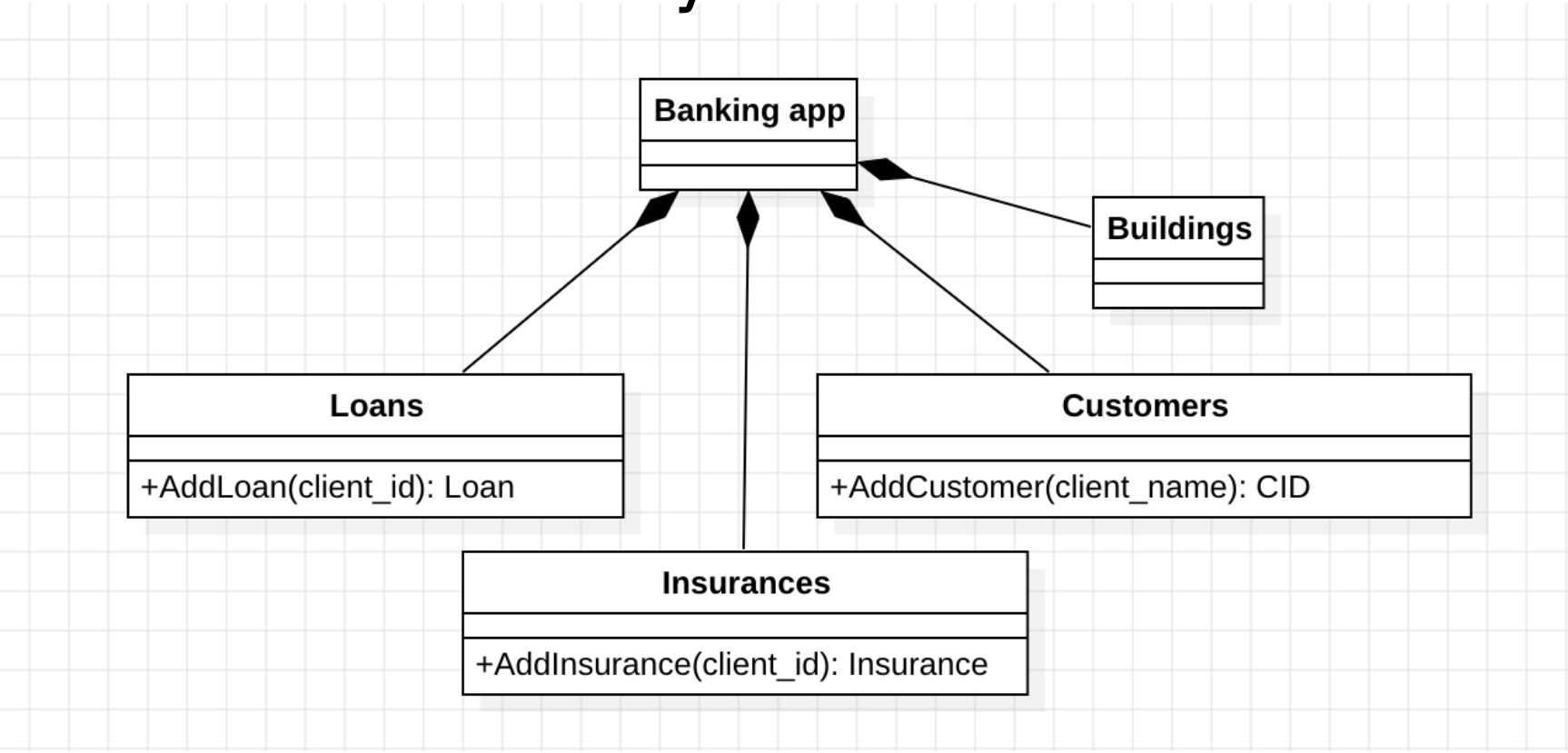
Design patterns need to be put in at the start

- But what if you have an existing app?
  - You did not design in the correct patterns
  - It just grew
    - Organically like, -ish, sort of

Example: Capislack bank

Banking app

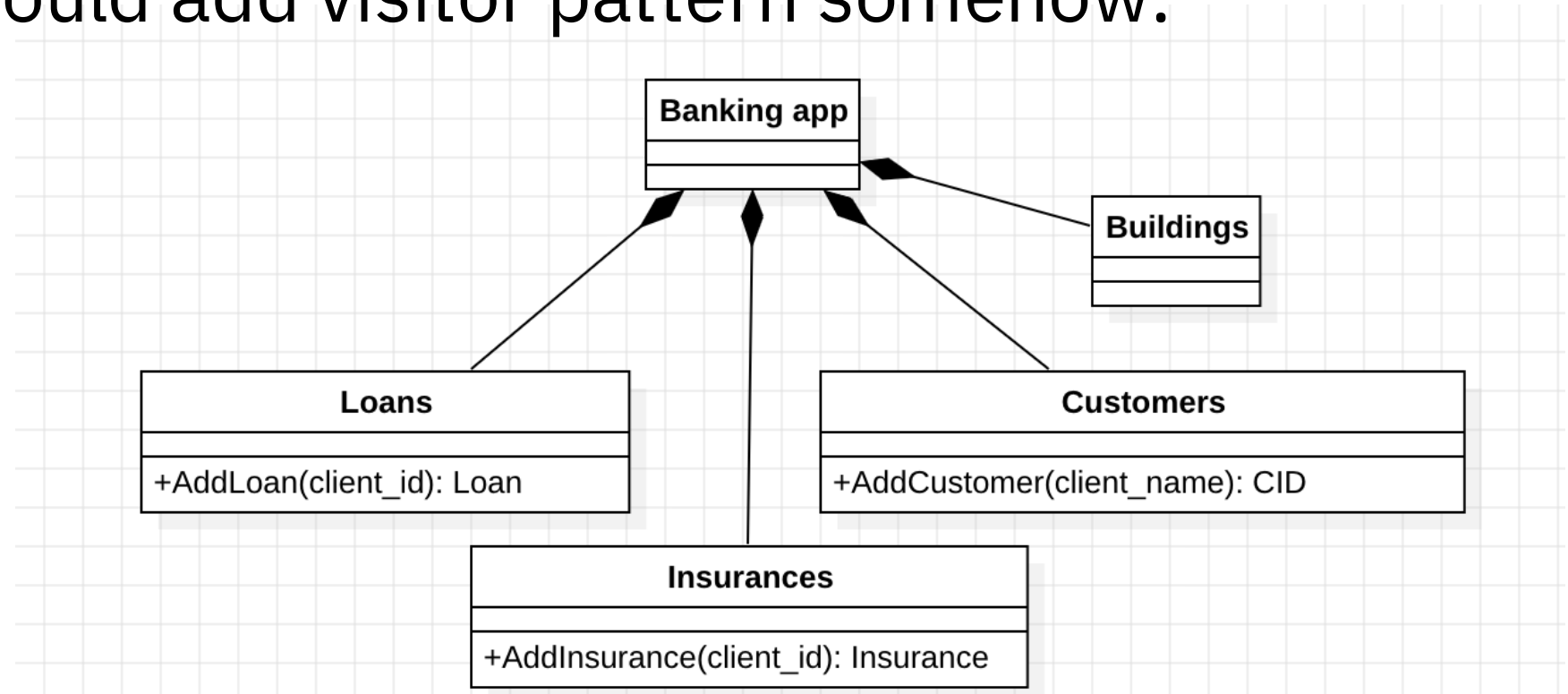
- Grew organically, over 40 years
- Now we need to add in security



Example: Capislack bank

Banking app

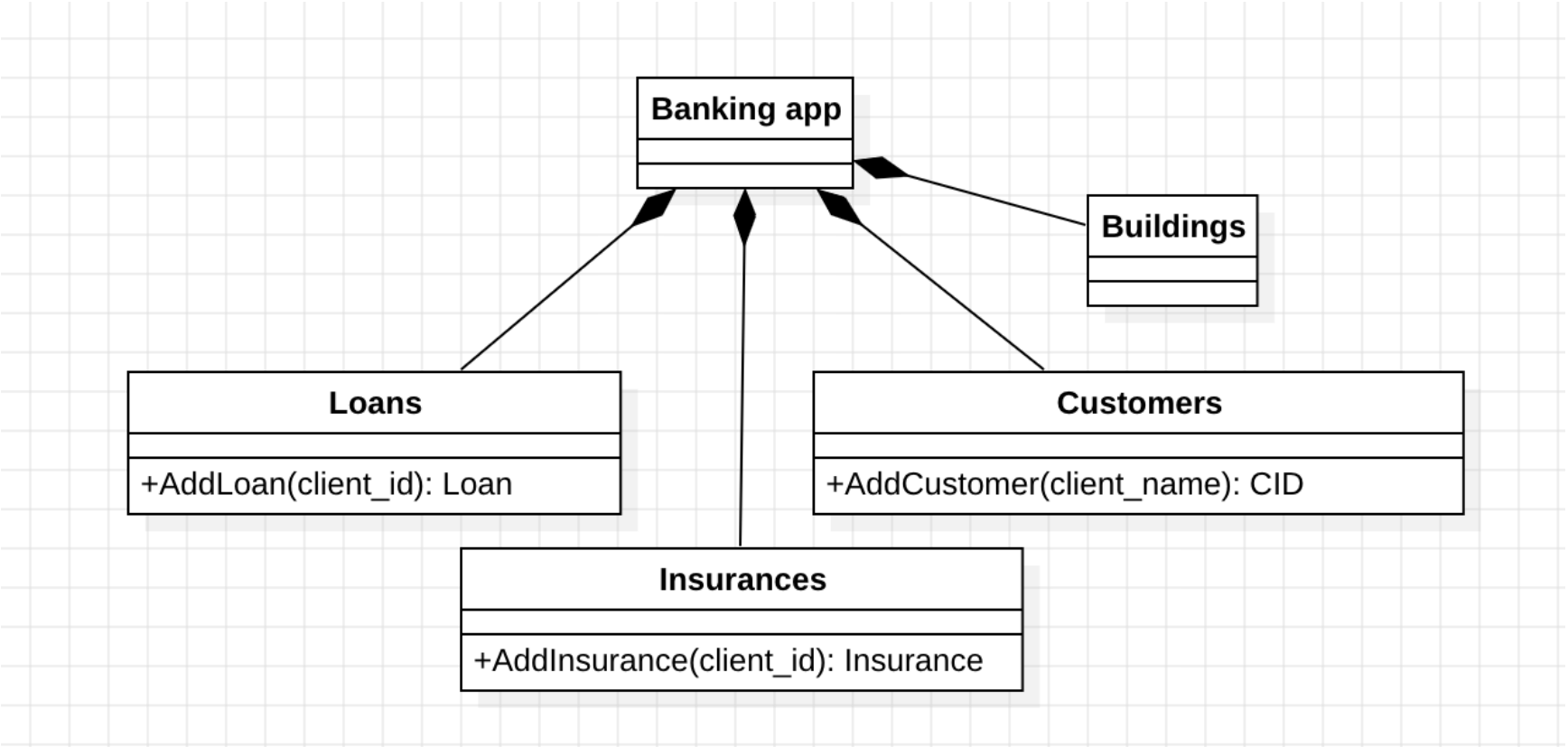
- Adding new features requires adding new operations
- Adding ops in existing OOP app is hard
  - If only we could add visitor pattern somehow.



Aspect oriented programming

Crosscutting concern examples:

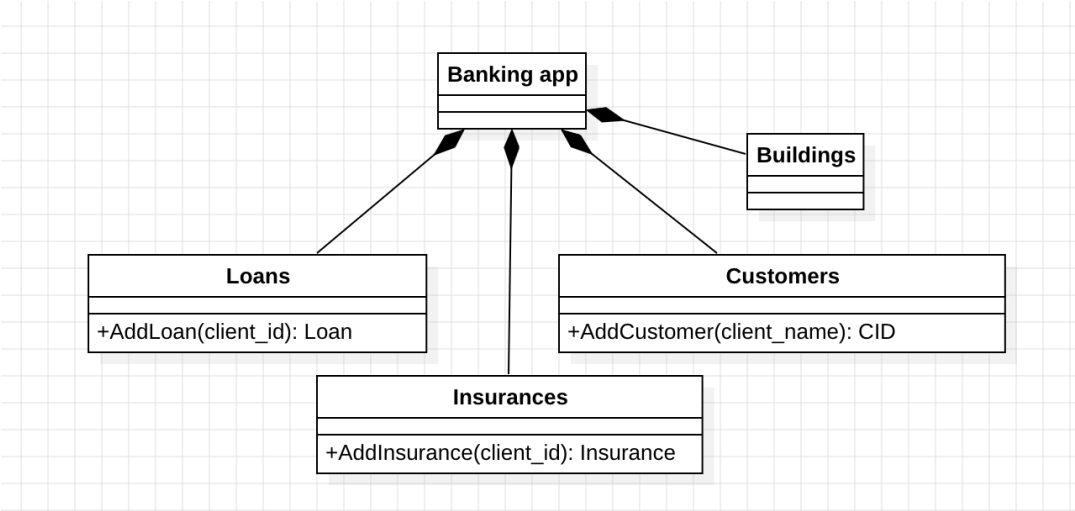
- new transactions only with approved clients
- new clients only if IDs proper & not banned



Aspect oriented programming

Crosscutting concerns:

- Different classes/interfaces need the same *intervention*
- Scattered methods have the same *requirements*
- Same concerns in different parts of application
  - cut across multiple different classes



Advice/Pointcut

Crosscutting concerns are fixed using an advice:

- **Advice**: a piece of code that is *injected* into 1 or more methods of your app.
- **Pointcut**: the location(s) that an advice is injected into
  - before method starts
  - after method ends
  - around the method
- **Aspect**: advice + pointcut

Pointcuts…

Pointcuts are defined using a regular expression

- Match on classname + function signature
  - e.g.: [A-Z]*s.Add*(client_id, *)
    - Match on class starting with capital, ends in 's' with a method that starts with 'Add' and has as its first parameter 'client_id'.
  - e.g. [A-Z]*s.Add*(client_id,*) | *.New*(*name, *)
    - Same, but also match any method starting with 'New' first parameter ends in 'name'
  - Matches can be and'ed (*must match all*) or or'ed (*can match any*) together

Advice

Advice code can run

- before: run prior starting method
- after: run after method has finished, ignore exceptions
- around: run part before, run method, then part after
- after exception: run only when exception happens
- after return: run after if no exceptions

Aspect = Pointcut + Advice

```
@Aspect
public class CheckNewTransactions {
    private Security sec;

    public CheckNewTransactions(Security sec){ this.sec = sec;}

    @Before("execution(* [A-Z]*s.Add*(client_id,..))")
    public void beforeMethodExecution(JoinPoint joinPoint) {
        Object[] args = joinPoint.getArgs();
        if (args[0] instanceof Integer) {
            final var client_id = (int)args[0];
            sec.check(is_customer(client_id));
            sec.check(!is_criminal(client_id));
            sec.check(!is_bankrupt(client_id));
```

How does it work?

Reflection: the ability of a process to examine, introspect, and **modify** its own structure and behavior

- Dynamic reflection: at runtime
  - Uses RTTI: runtime type information
  - Aka: Injection
- Static reflection: at compile time
  - *The compiler does 'search + replace' on source code*
  - Aka: instrumentation

AspectJ

Java AOP

- Since 2001
- Instruments source code using an extra compile step
- Using the AspectJ *weaving* compiler

Pros and cons

Pro	Con
Modularity: Can easily add crosscutting operations	**Debugging**: class methods only contain part of the code, crosscuts can happen at any time
Code reusability: Makes DRY code easy (apply one aspect in many places)	**Complexity**: Easy to weave very complex crosscuts (Adds an extra dimension to your hierarchy)
Maintainability: easy to patch existing code (even if existing design sub-optimal)	**Tooling**: IDE may make crosscuts hard to see
SOLID: reduces dependencies, separates crosscutting concerns from main logic.	**Portability**: AOP not portable across languages Java has the best support, C++ not yet (maybe C++29)
Scalability: OAP based crosscuts are lightweight	
Testability: Testing simplified because DRY'er code	

DEMO